BÁO CÁO THỰC HIỆN

CHẠY THỬ MODEL AI TRÊN THIẾT BỊ ESP32

Nền tảng: ESP32-S3 (N16R8) – PlatformIO / Arduino Framework

Model ứng dụng: Phân loại rác thải (Garbage Classification)

Các lớp phân loại: Battery · Biological · Cardboard · Metal · Paper

Thư viện suy diễn: `TensorFlowLite_ESP32` (LiteRT) & `AIoT_Edge_Impulse`

Ngày 6 tháng 4 năm 2026

Mục lục

1 Tổng quan hệ thống **2**

2 Cập nhật Web Server – Truyền ảnh qua WebSocket **3**

2.1 Giao thức truyền ảnh 3

2.2 Lý do chia chunk và xử lý phía trình duyệt 4

3 Module ghép ảnh và chuyển đổi Pixel (`image_concat`) **5**

3.1 Sơ đồ pipeline xử lý ảnh 6

3.2 Bước 1 – Ghép chunk và giải mã Base64 6

3.3 Bước 2 – Giải mã JPEG thành pixel RGB888 6

3.4 Bước 3+4 – Resize và Quantize INT8 7

3.5 Quản lý buffer đôi và cờ trạng thái 7

4 Suy diễn với TensorFlow Lite Micro – Thư viện `TensorFlowLite_ESP32` **8**

4.1 Giới thiệu thư viện 8

4.2 Chuyển sang LiteRT (phiên bản mới) 8

4.3 Cấu hình `platformio.ini` 9

4.4 Luồng khởi tạo và suy diễn 10

4.5 Vấn đề tensor arena và PSRAM 10

4.6 Giải quantize kết quả đầu ra 11

5 Suy diễn với Edge Impulse – Thư viện `AIoT_Edge_Impulse` **11**

5.1 Giới thiệu thư viện 11

5.2 Các thay đổi để tương thích với dự án 12

5.3 Luồng suy diễn Edge Impulse 14

5.4 Kết quả đầu ra 14

6 Tổng kết **15**

7 Đánh giá hiệu năng thực tế trên ESP32 **15**

7.1 Thiết lập kiểm thử 15

7.2 Kết quả thô 16

7.3 Ma trận nhầm lẫn (Confusion Matrix) 17

7.4 Chỉ số đánh giá theo lớp 17

7.5 Phân tích kết quả 18

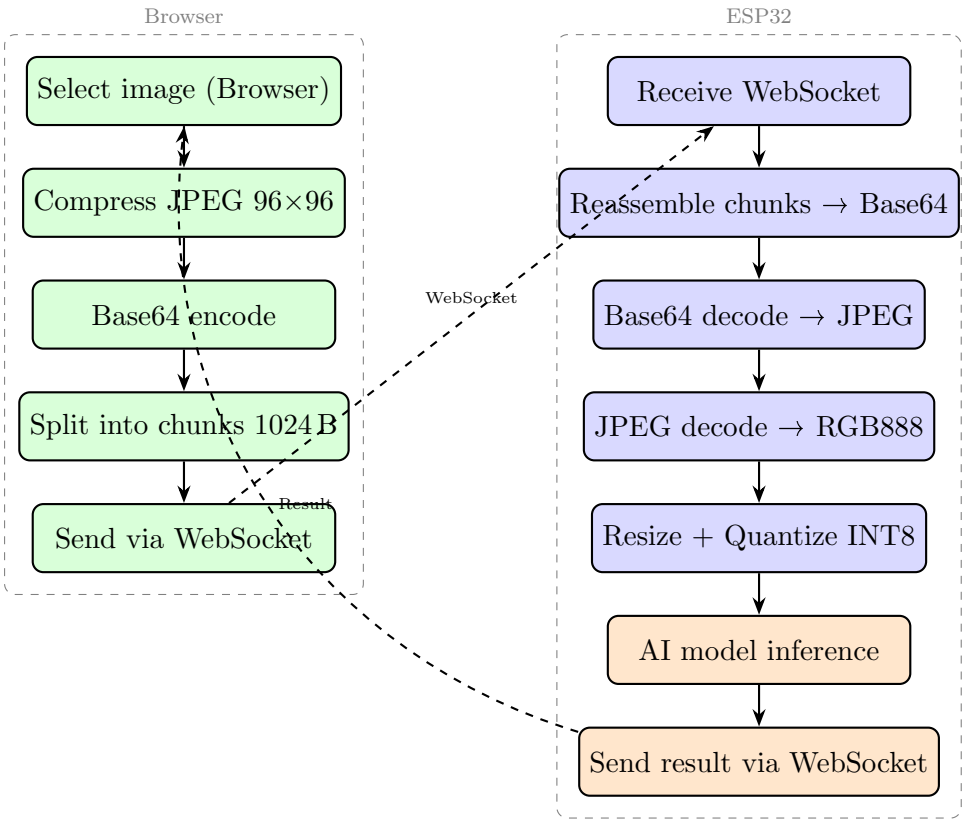
1 Tổng quan hệ thống

Hệ thống thực hiện chu trình từ đầu đến cuối: người dùng chọn ảnh trên trình duyệt, ảnh được nén và chia nhỏ rồi truyền tới ESP32 thông qua WebSocket; vi điều khiển giải nén, chuyển đổi định dạng pixel, sau đó chạy mô hình AI để phân loại và trả kết quả về trang web.

Mục tiêu thiết kế chính:

- Không yêu cầu camera gắn thêm – ảnh được gửi từ thiết bị di động / máy tính.
- Tận dụng bộ nhớ PSRAM 8 MB (SPIRAM) trên ESP32-S3-N16R8 để chứa tensor arena.
- Hỗ trợ hai back-end suy diễn có thể chuyển đổi qua biên dịch: `TFLite Micro` và `Edge Impulse EON`.

Sơ đồ luồng tổng thể



Hình 1: End-to-end processing flow from browser to ESP32

2 Cập nhật Web Server – Truyền ảnh qua WebSocket

2.1 Giao thức truyền ảnh

Web server trên ESP32 sử dụng thư viện `ESPAsyncWebServer` với điểm cuối WebSocket tại địa chỉ `/ws`. Trước khi tích hợp tính năng AI, WebSocket chỉ xử lý các trang `setting` và `device` (điều khiển relay, cấu hình WiFi). Để hỗ trợ truyền ảnh, một trang mới `ai_detect` được thêm vào với ba kiểu thông điệp:

Bảng 1: Các kiểu thông điệp WebSocket cho trang `ai_detect`

type	Chiều	Mô tả
`image_start`	Browser → ESP32	Báo bắt đầu truyền, kèm `imageId` và `totalChunks`
`image_chunk`	Browser → ESP32	Một phần dữ liệu Base64, kèm `index` và `chunk`
`image_end`	Browser → ESP32	Báo kết thúc, ESP32 bắt đầu xử lý ảnh

Đoạn xử lý trong `web_server.cpp`:

```
if (page == "ai_detect") {
    if (type == "image_start") {
        String imageId = recvDoc["value"]["imageId"].as<String>();
        int totalChunks = recvDoc["value"]["totalChunks"].as<int>()
    ;
        imageConcatStart(imageId, totalChunks);
        return;
    }
    if (type == "image_chunk") {
        String imageId = recvDoc["value"]["imageId"].as<String>();
        int index      = recvDoc["value"]["index"].as<int>();
        String chunk   = recvDoc["value"]["chunk"].as<String>();
        imageConcatAppendChunk(imageId, index, chunk);
        return;
    }
    if (type == "image_end") {
        String imageId = recvDoc["value"]["imageId"].as<String>();
        imageConcatFinish(imageId);
        return;
    }
}
```

Listing 1: Xử lý trang `ai_detect` trong `requestProcessing()`

2.2 Lý do chia chunk và xử lý phía trình duyệt

Tại sao phải chia nhỏ dữ liệu?

Giao thức WebSocket không giới hạn độ lớn của một khung dữ liệu về mặt lý thuyết, nhưng trên thực tế tồn tại hai giới hạn quan trọng:

1. **Bộ nhớ stack của ESP32:** Thư viện `ESPAsyncWebServer` xử lý dữ liệu WebSocket trong callback ngắt. Việc nhận một chuỗi Base64 có kích thước 5–10 KB trong một lần cấp phát tĩnh gây mất ổn định heap trên ESP32 (heap fragmentation), đặc biệt khi có nhiều task FreeRTOS đang chạy đồng thời.

2. **Giới hạn kích thước frame:** `ESPAsyncWebServer` xử lý WebSocket message theo từng frame. Nếu trình duyệt gửi một message quá lớn, thư viện có thể phân mảnh nội bộ và gây lỗi parsing.

Giải pháp là phía JavaScript cắt chuỗi Base64 thành các đoạn (chunk) 1024 byte, gửi tuần tự:

```javascript
const IMAGE_CHUNK_SIZE = 1024;

async function sendCompressedImageChunked(file) {
    const compressedBlob = await compressImageFile(
                                file, IMAGE_MAX_WIDTH, IMAGE_MAX_HEIGHT,
                                IMAGE_JPEG_QUALITY);          // 96x96, quality 0.65
    const base64Payload = await blobToBase64String(compressedBlob);
    const imageId       = `${Date.now()}_${Math.random().toString(16).slice(2,8)}`;
    const totalChunks   = Math.ceil(base64Payload.length / IMAGE_CHUNK_SIZE);

    // 1. Send start message
    Send_Data(JSON.stringify({ page:'ai_detect', type:'image_start',
        value:{ imageId, totalChunks } }));

    // 2. Send each chunk
    for (let i = 0; i < totalChunks; i++) {
        const chunk = base64Payload.slice(i * IMAGE_CHUNK_SIZE,
                                          (i+1) * IMAGE_CHUNK_SIZE);
        Send_Data(JSON.stringify({ page:'ai_detect', type:'image_chunk',
            value:{ imageId, index: i, chunk } }));
        await sleep(0); // yield to event loop -- avoid blocking UI
```

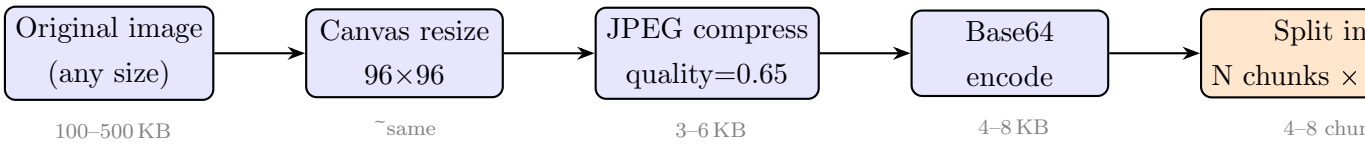
```
    }

    // 3. Send end message
    Send_Data(JSON.stringify({ page:'ai_detect', type:'image_end',
        value:{ imageId, totalChunks } }));
}
```

Listing 2: Gửi ảnh theo chunk trong `script.js`

Nén ảnh trước khi gửi

Trước khi chia chunk, ảnh gốc được thu nhỏ về kích thước đầu vào của mô hình (96×96 pixel) và nén thành JPEG với chất lượng 0.65 ngay tại trình duyệt. Điều này giảm kích thước dữ liệu cần truyền từ hàng chục KB (ảnh gốc) xuống còn khoảng 3–6 KB (JPEG 96×96), tương đương chỉ 3–6 chunk.

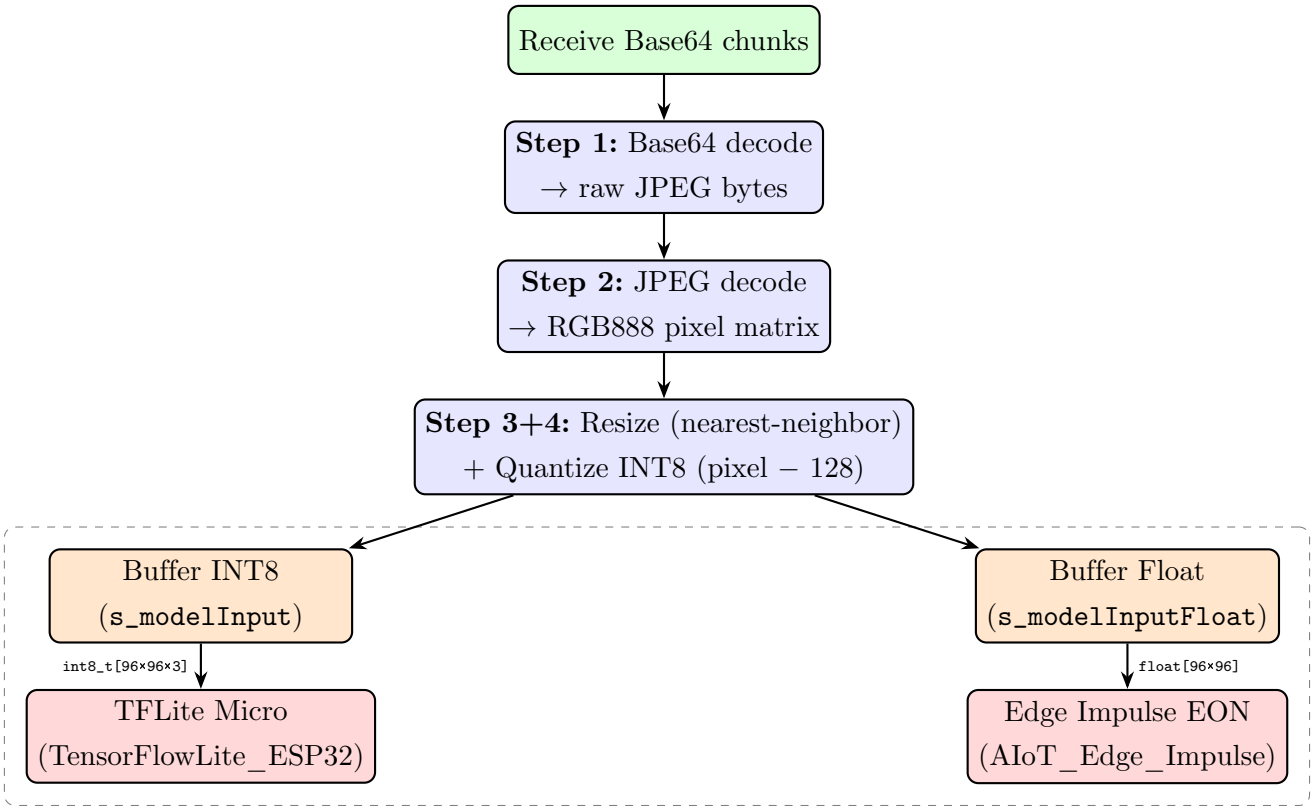


Hình 2: Browser-side image processing pipeline before transmission

3 Module ghép ảnh và chuyển đổi Pixel (`image_concat`)

Module `image_concat.cpp / .hpp` đảm nhận toàn bộ pipeline xử lý trung gian: từ khi nhận chunk đầu tiên cho đến khi tensor đầu vào cho mô hình AI đã sẵn sàng.

3.1 Sơ đồ pipeline xử lý ảnh



Hình 3: Image processing pipeline in the `image_concat` module

3.2 Bước 1 – Ghép chunk và giải mã Base64

Khi `imageConcatAppendChunk()` được gọi, module nối chuỗi chunk vào một `String` dự trữ (`base64Payload`). Sau khi nhận đủ `totalChunks` chunk và hàm `imageConcatFinish()` được gọi, toàn bộ chuỗi Base64 được giải mã thành byte JPEG thô bằng hàm `mbedtls_base64_decode` – thư viện mbedTLS đã được tích hợp sẵn trong ESP32 Arduino framework, không cần cài thêm.

Buffer trung gian `s_jpegBuf` được khai báo tĩnh với kích thước 24 KB – đủ để chứa JPEG 96×96 ở chất lượng 0.65.

3.3 Bước 2 – Giải mã JPEG thành pixel RGB888

Thư viện **JPEGDEC** (bitbank2) được dùng để giải mã JPEG hoàn toàn trong RAM mà không cần filesystem. Chế độ `RGB8888` được chọn (thay vì RGB565) để giữ đầy đủ 8 bit mỗi kênh màu, tránh sai số làm tròn khi quantize sau này.

Điểm quan trọng: JPEGDEC trả về pixel theo định dạng little-endian `0xFFRRGGBB`, tức là thứ tự byte trong bộ nhớ là [`B, G, R, 0xFF`] – **4 byte** mỗi pixel. Hàm callback `jpegDrawCallback()` phải đọc đúng thứ tự này:

```
static int jpegDrawCallback(JPEGDRAW *pDraw) {
    const uint8_t *src = (const uint8_t *)pDraw->pPixels;
    for (int row = 0; row < pDraw->iHeight; row++) {
        for (int col = 0; col < pDraw->iWidth; col++) {
            int dstBase = (dstY * MODEL_INPUT_W + dstX) * 3;
            int srcBase = (row  * pDraw->iWidth  + col)  * 4; // 4 byte/pixel!
            s_pixelBuf[dstBase + 0] = src[srcBase + 2]; // R
            s_pixelBuf[dstBase + 1] = src[srcBase + 1]; // G
            s_pixelBuf[dstBase + 2] = src[srcBase + 0]; // B
        }
    }
    return 1;
}
```

Listing 3: Callback JPEGDEC – chú ý thứ tự byte RGB8888

3.4 Bước 3+4 – Resize và Quantize INT8

Vì phía trình duyệt đã resize ảnh về đúng 96×96 trước khi gửi, bước resize phía ESP32 thực chất là một phép copy thẳng (tỷ lệ = 1.0). Tuy nhiên code vẫn giữ logic nearest-neighbor để an toàn khi nhận ảnh có kích thước khác.

Quantize từ `uint8` [0, 255] sang `int8` [-128, 127]:

$$x_{\text{int8}} = x_{\text{uint8}} - 128$$

Cách này tương đương chuẩn hóa MobileNet float $x' = x/127.5 - 1.0$ với scale = 1/128, zero_point = 0, và tránh phải cấp phát thêm buffer float.

Đồng thời, buffer float cho Edge Impulse cũng được điền đồng thời tại đây theo định dạng *packed-RGB*:

$$v_{\text{float}} = (\text{R} \ll 16) \mid (\text{G} \ll 8) \mid \text{B}$$

trong đó R, G, B là giá trị INT8 ngay vừa tính.

3.5 Quản lý buffer đôi và cờ trạng thái

Hai cờ tĩnh `s_newImageReadyInt` và `s_newImageReadyFloat` được dùng để thông báo cho hai task suy diễn độc lập. Mỗi task chỉ xoá cờ của riêng mình, không ảnh hưởng đến task còn lại. Điều này cho phép cả TFLite Micro và Edge Impulse cùng tiêu thụ cùng một ảnh mà không cần copy buffer.

4 Suy diễn với TensorFlow Lite Micro – Thư viện `TensorFlowLite_ESP32`

4.1 Giới thiệu thư viện

`TensorFlowLite_ESP32` (trong `lib/TensorFlowLite_ESP32/`) là một port của TensorFlow Lite for Microcontrollers (TFLM) dành riêng cho ESP32 Arduino framework, được đóng gói dưới dạng thư viện PlatformIO/Arduino. Phiên bản trong dự án này dựa trên codebase TFLM cũ (pre-LiteRT), sử dụng namespace `tflite::` với các header quen thuộc như `tensorflow/lite/micro/micro_interpreter.h`.

4.2 Chuyển sang LiteRT (phiên bản mới)

Kể từ năm 2024, Google đổi tên TensorFlow Lite thành **LiteRT** và tái cấu trúc toàn bộ kho mã nguồn tại `https://github.com/tensorflow/tflite-micro`. Thư viện mới không còn được ship sẵn dưới dạng file nén cho Arduino; thay vào đó, người dùng phải chạy script Python để sinh ra cây thư mục nguồn phù hợp với target (xem file `ESP32_PROJECT_GENERATION.md`).

Điểm khác biệt chính giữa hai phiên bản:

Bảng 2: So sánh `TensorFlowLite_ESP32` (cũ) và LiteRT (mới)

Khía cạnh	**TensorFlowLite_ESP32 (cũ)**	**LiteRT / tflite-micro (mới)**
Phân phối	Thư viện PlatformIO/Arduino đóng gói sẵn	Sinh từ script Python, đặt trong `lib/`
Phiên bản schema	Cố định (cũ)	Cập nhật thường xuyên
Phần mở rộng file	Đã đổi `.cc` → `.cpp` sẵn	Cần flag `-rename_cc_to_cpp` khi sinh
Namespace API	`tflite::`	`tflite::` (giữ nguyên)
Tích hợp	Thêm vào `lib_deps`	Copy vào `lib/` + thêm include path
Hỗ trợ PSRAM	Cần patch thủ công	Hỗ trợ tốt, arena có thể cấp phát từ PSRAM

Quy trình sinh thư viện LiteRT cho ESP32 (tóm tắt từ `ESP32_PROJECT_GENERATION.md`):

```
# Step 1: Download third-party dependencies (flatbuffers, kissfft,
    ruy, gemmlowp)
cd tflite-micro
```

```
make -f tensorflow/lite/micro/tools/make/Makefile
    third_party_downloads

# Step 2: Generate source tree
python3 tensorflow/lite/micro/tools/project_generation/
    create_tflm_tree.py \
  --rename_cc_to_cpp \
  --examples hello_world \
  /path/to/output/tflm_esp32

# Step 3: Copy into PlatformIO project
cp -r tflm_esp32/ lib/TensorFlowLite_ESP32/
```

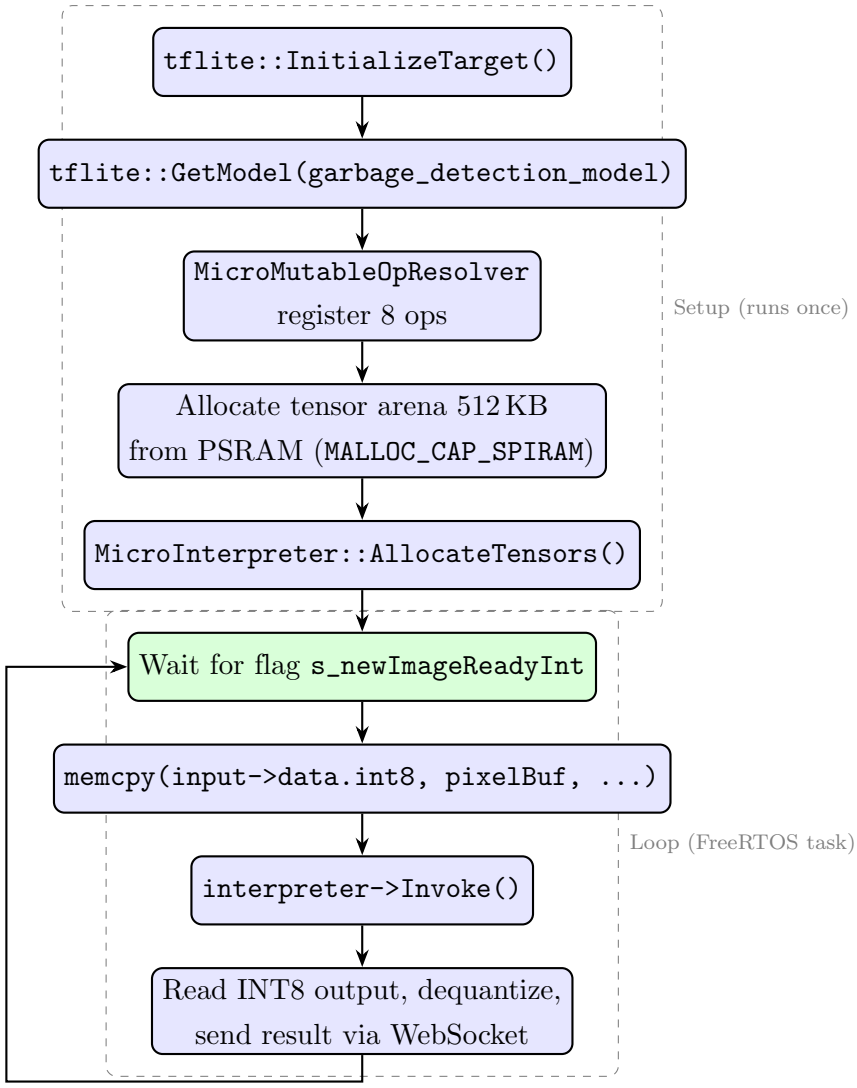
Listing 4: Sinh cây thư mục LiteRT cho ESP32

4.3 Cấu hình `platformio.ini`

```
[env:tinyml]
build_flags =
    ${common.build_flags}
    -D ENABLE_TINYML
    -I lib/TensorFlowLite_ESP32/src/third_party/flatbuffers/include
    -I lib/TensorFlowLite_ESP32/src/third_party/gemmlowp
    -I lib/TensorFlowLite_ESP32/src/third_party/ruy
    -I lib/TensorFlowLite_ESP32/src/third_party/kissfft
lib_deps =
    ${common.lib_deps}
    TensorFlowLite_ESP32
lib_ignore = AIoT_Edge_Impulse   ; avoid TFLite symbol conflict
```

Listing 5: Cấu hình PlatformIO cho back-end TFLite Micro

4.4 Luồng khởi tạo và suy diễn



Hình 4: TFLite Micro initialization and inference flow

4.5 Vấn đề tensor arena và PSRAM

Model phân loại rác (MobileNetV2 96×96) có lớp Conv2D đầu tiên tạo ra activation tensor 96×96×32 = 288 KB. Bộ nhớ SRAM nội bộ của ESP32-S3 chỉ khoảng 520 KB tổng cộng, không thể chứa đủ các tensor trung gian. Do đó, tensor arena được cấp phát 512 KB từ PSRAM:

```
tensor_arena = (uint8_t *)heap_caps_malloc(
    kTensorArenaSize,
    MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT
);
```

Listing 6: Cấp phát tensor arena từ PSRAM

4.6 Giải quantize kết quả đầu ra

Model đầu ra là INT8 với scale và zero_point được lưu trong `output->params`:

```
float score = (output->data.int8[i] - output->params.zero_point)
                * output->params.scale;
```

Listing 7: Giải quantize giá trị INT8 ra xác suất float

Kết quả là xác suất [0.0, 1.0] cho mỗi trong 5 nhãn: *Battery, Biological, Cardboard, Metal, Paper*.

5 Suy diễn với Edge Impulse – Thư viện `AIoT_Edge_Impulse`

5.1 Giới thiệu thư viện

`AIoT_Edge_Impulse` (trong `lib/AIoT_Edge_Impulse/`) là một *C++ deployment library* được sinh tự động bởi nền tảng **Edge Impulse Studio**. Thư viện đóng gói cả pipeline DSP, model (*EON compiled* – mã C++ sinh từ TFLite), và metadata của impulse vào một package duy nhất. Người dùng chỉ cần gọi hàm `run_classifier()` để thực hiện toàn bộ chuỗi DSP + suy diễn.

So với `TensorFlowLite_ESP32`, thư viện Edge Impulse có những điểm khác biệt chính:

Bảng 3: So sánh hai back-end suy diễn

Khía cạnh	**TensorFlowLite_ESP32**	**AIoT_Edge_Impulse**
Nguồn gốc	Port thủ công từ TFLM	Sinh tự động từ Edge Impulse Studio
API chính	`interpreter->Invoke()`	`run_classifier(&signal, &result)`
Quản lý tensor	Người dùng tự quản lý	Nội bộ tự động (qua `ei_impulse_handle_t`)
Định dạng đầu vào	INT8 thô `[H][W][C]`	Float packed RGB `(R«16|G«8|B)`
Tích hợp DSP	Không	Có (resize, normalization tự động)
Model lưu trữ	File `.h` (flat byte array)	EON compiled `tflite_learn_943952_6_compiled.cpp`
Kết quả đầu ra	Mảng INT8 cần giải quantize thủ công	Struct `ei_impulse_result_t` với xác suất float sẵn

Thư viện trong dự án tương ứng với project **NamDang_AIoT** (ID: 943952) trên Edge Impulse Studio, với các tham số:

- Đầu vào: 96×96 RGB (9216 pixel, `EI_CLASSIFIER_RAW_SAMPLE_COUNT = 9216`)
- DSP block: Image – Fit Shortest
- Số lớp đầu ra: 5 (`EI_CLASSIFIER_NN_OUTPUT_COUNT = 5`)
- Model: TFLite INT8 EON compiled

5.2 Các thay đổi để tương thích với dự án

Việc tích hợp thư viện Edge Impulse vào dự án PlatformIO không phải là plug-and-play. Một số thay đổi đã được thực hiện:

1. Tách biệt với TFLite backend để tránh xung đột

Cả `TensorFlowLite_ESP32` lẫn `AIoT_Edge_Impulse` đều đóng gói bản sao của TFLite Micro. Nếu cả hai cùng được biên dịch sẽ sinh lỗi *multiple definition*. Giải pháp là dùng macro `#ifdef` cùng với `lib_ignore` trong `platformio.ini`:

```
[env:ei_classifier]
build_flags =
    ${common.build_flags}
    -D ENABLE_EI_CLASSIFIER
lib_deps =
    ${common.lib_deps}
    ; AIoT_Edge_Impulse is used from local lib/ folder
lib_ignore = TensorFlowLite_ESP32  ; avoid TFLite symbol conflict

[env:tinyml]
lib_ignore = AIoT_Edge_Impulse
```

Listing 8: `platformio.ini` – loại trừ thư viện xung đột

Trong mã nguồn C++:

```
// ei_inference.cpp
#ifdef ENABLE_EI_CLASSIFIER
  #include "edge-impulse-sdk/classifier/ei_run_classifier.h"
  // ... all Edge Impulse inference logic
#endif

// tinyml_img.cpp
#ifdef ENABLE_TINYML
  #include <TensorFlowLite_ESP32.h>
  // ... all TFLite Micro inference logic
```

```
#endif
```

Listing 9: Guard macro để tách biệt hai back-end

2. Định dạng đầu vào signal_t

Edge Impulse sử dụng callback `signal_t` để cung cấp dữ liệu pixel theo yêu cầu:

```
int raw_feature_get_data(size_t offset, size_t length, float *
    out_ptr) {
     memcpy(out_ptr, pixelBuf + offset, length * sizeof(float));
     return 0;
}

signal_t features_signal;
features_signal.total_length = EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE;
features_signal.get_data     = &raw_feature_get_data;

EI_IMPULSE_ERROR res = run_classifier(&features_signal, &result,
    false);
```

Listing 10: Cài đặt signal_t cho Edge Impulse

Buffer `pixelBuf` trỏ thẳng vào `s_modelInputFloat` từ module `image_concat` – không cần sao chép dữ liệu.

3. Kiểm tra kích thước đầu vào

Trước khi gọi `run_classifier()`, cần kiểm tra số lượng pixel khớp với metadata của model:

```
if (numValues != EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE) {
     Serial.printf("Mismatch: expected=%d got=%d\n",
                    EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE, (int)
    numValues);
     continue;
}
```

Listing 11: Kiểm tra kích thước đầu vào

`EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE = 9216` = 96×96 (một float packed-RGB cho mỗi pixel).

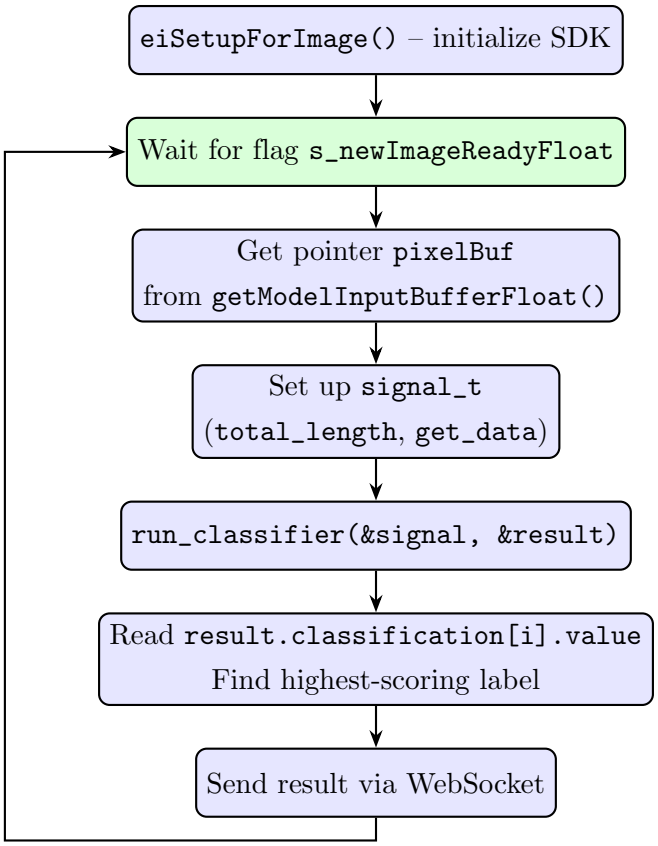
4. Watchdog Timer

Suy diễn model ảnh float32 trên ESP32 có thể mất 20–40 giây. Task FreeRTOS phải tự xoá khỏi Watchdog Timer để tránh bị reset:

```
#include "esp_task_wdt.h"
// ...
esp_task_wdt_delete(NULL); // call before the while(1) loop
```

Listing 12: Tắt Watchdog cho task suy diễn nặng

5.3 Luồng suy diễn Edge Impulse



Hình 5: Edge Impulse inference flow in FreeRTOS task

5.4 Kết quả đầu ra

```
// Print timing
ei_printf("DSP: %d ms | Inference: %d ms\n",
    result.timing.dsp, result.timing.classification);

// Find label with highest probability
for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (result.classification[i].value > max_score) {
        max_score = result.classification[i].value;
        max_label = ei_classifier_inferencing_categories[i];
    }
}
```

Listing 13: Đọc kết quả từ `ei_impulse_result_t`

Kết quả được gửi qua WebSocket về trình duyệt dưới dạng JSON:

```
{
  "page": "ai_detect",
  "value": {
    "label": "cardboard",
    "score": "0.924",
    "time":  "1350"
  }
}
```

Listing 14: JSON kết quả trả về trình duyệt

6 Tổng kết

Dự án đã hiện thực thành công hai back-end suy diễn AI trên ESP32-S3 với các điểm nổi bật:

1. **Truyền ảnh qua WebSocket chunked:** Giải quyết giới hạn bộ nhớ ESP32 khi nhận dữ liệu lớn. Protocol ba bước (start/chunk/end) đảm bảo tính toàn vẹn dữ liệu và dễ mở rộng.

2. **Pipeline xử lý ảnh trung tâm:** Module `image_concat` đảm nhận toàn bộ quá trình từ Base64 đến tensor đầu vào, cung cấp hai buffer độc lập (INT8 và float) cho hai back-end tiêu thụ mà không cần sao chép dữ liệu.

3. **TFLite Micro (LiteRT):** Phù hợp khi cần kiểm soát hoàn toàn pipeline và model format. Đòi hỏi quản lý tensor, ops thủ công và cần nâng cấp thư viện từ script Python của tflite-micro.

4. **Edge Impulse EON:** API đơn giản hơn, bao gồm DSP tích hợp và kết quả float sẵn. Model được sinh tự động từ Edge Impulse Studio, dễ cập nhật khi train lại.

Cả hai back-end được tách biệt hoàn toàn qua macro biên dịch, có thể chuyển đổi bằng cách thay giá trị `default_envs` trong `platformio.ini`.

7 Đánh giá hiệu năng thực tế trên ESP32

7.1 Thiết lập kiểm thử

Bộ dữ liệu kiểm thử gồm **20 ảnh** (4 ảnh mỗi lớp) thuộc 5 lớp phân loại rác: *Battery, Biological, Cardboard, Metal, Paper*. Mỗi ảnh được nén về 96×96 JPEG tại trình duyệt

rồi gửi qua WebSocket đến ESP32-S3. Hai back-end chạy độc lập trên cùng thiết bị và cùng ảnh đầu vào.

Lưu ý: Kết quả của Edge Impulse dưới đây là sau khi sửa lỗi packed-RGB trong hàm `resizeAndNormalize()` (Module `image_concat`). Lần kiểm thử đầu tiên (trước khi sửa) cho accuracy chỉ 45% do dữ liệu pixel bị mã hoá sai.

7.2 Kết quả thô

Bảng 4: Kết quả phân loại thô của từng mẫu kiểm thử

Mẫu	**Edge Impulse (EON)**			**LiteRT (TFLite Micro)**		
	Dự đoán	Conf (%)	Time (ms)	Dự đoán	Conf (%)	Time (ms)
battery1	**battery**	99.60	211	**Battery**	51.95	6455
battery2	**battery**	98.82	203	Metal	71.48	6452
battery3	biological	90.234	201	Metal	81.25	6458
battery4	biological	39.06	203	Metal	50.39	6454
biological1	**biological**	52.34	203	**Biological**	71.48	6452
biological2	**biological**	99.61	206	**Biological**	95.70	6461
biological3	**biological**	54.30	203	Metal	70.31	6461
biological4	**biological**	93.36	204	**Biological**	80.08	6452
cardboard1	**cardboard**	94.92	202	**Cardboard**	98.83	6447
cardboard2	**cardboard**	68.36	201	Battery	32.81	6454
cardboard3	**cardboard**	88.28	204	**Cardboard**	60.16	6448
cardboard4	biological	99.22	209	**Cardboard**	42.97	6454
metal1	**metal**	98.05	202	**Metal**	99.21	6454
metal2	**metal**	88.28	204	**Metal**	96.48	6454
metal3	**metal**	85.55	202	**Metal**	97.27	6457
metal4	**metal**	99.61	203	**Metal**	99.61	6454
paper1	**paper**	99.61	202	Cardboard	67.97	6449
paper2	**paper**	99.61	202	**Paper**	54.68	6447
paper3	cardboard	82.03	201	**Paper**	87.89	6449
paper4	**paper**	95.31	202	**Paper**	42.19	6446
Avg time (ms)			**203.4**			**6452.9**
Accuracy		**16/20 = 80.0%**			**14/20 = 70.0%**	

Các ô in đậm biểu thị dự đoán **đúng** với nhãn thật.

7.3 Ma trận nhầm lẫn (Confusion Matrix)

Bảng 5: Confusion matrix – Edge Impulse EON (hàng = nhãn thật, cột = dự đoán)

True \ Pred	*battery*	*biological*	*cardboard*	*metal*	*paper*	**Total**
battery	**2**	2	0	0	0	4
biological	0	**4**	0	0	0	4
cardboard	0	1	**3**	0	0	4
metal	0	0	0	**4**	0	4
paper	0	0	1	0	**3**	4
Total pred	2	7	4	4	3	20

Bảng 6: Confusion matrix – LiteRT / TFLite Micro (hàng = nhãn thật, cột = dự đoán)

True \ Pred	*battery*	*biological*	*cardboard*	*metal*	*paper*	**Total**
battery	**1**	0	0	3	0	4
biological	0	**3**	0	1	0	4
cardboard	1	0	**3**	0	0	4
metal	0	0	0	**4**	0	4
paper	0	0	1	0	**3**	4
Total pred	2	3	4	8	3	20

7.4 Chỉ số đánh giá theo lớp

Precision, Recall và F1-score được tính theo công thức chuẩn:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad \text{F1}_c = \frac{2 \cdot P_c \cdot R_c}{P_c + R_c}$$

Bảng 7: Chỉ số per-class – Edge Impulse EON (sau khi sửa lỗi pixel buffer)

Lớp	**TP**	**FP**	**FN**	**Precision**	**Recall**	**F1**
battery	2	0	2	1.000	0.500	0.667
biological	4	3	0	0.571	1.000	0.727
cardboard	3	1	1	0.750	0.750	0.750
metal	4	0	0	1.000	1.000	1.000
paper	3	0	1	1.000	0.750	0.857
Macro avg				**0.864**	**0.800**	**0.800**

Bảng 8: Chỉ số per-class – LiteRT / TFLite Micro

Lớp	**TP**	**FP**	**FN**	**Precision**	**Recall**	**F1**
battery	1	1	3	0.500	0.250	0.333
biological	3	0	1	1.000	0.750	0.857
cardboard	3	1	1	0.750	0.750	0.750
metal	4	4	0	0.500	1.000	0.667
paper	3	0	1	1.000	0.750	0.857
Macro avg				**0.750**	**0.700**	**0.693**

Bảng 9: So sánh tổng hợp hai back-end

Chỉ số	**Edge Impulse EON**	**LiteRT (TFLite Micro)**
Accuracy	**80.0%**	70.0%
Macro Precision	**0.864**	0.750
Macro Recall	**0.800**	0.700
Macro F1	**0.800**	0.693
Avg Inference	**203.4 ms**	6452.9 ms

7.5 Phân tích kết quả

Tác động của việc sửa lỗi pixel buffer

Trước khi sửa lỗi packed-RGB trong `image_concat.cpp`, Edge Impulse chỉ đạt accuracy **45%**. Sau khi sửa, accuracy tăng lên **80%**, vượt LiteRT (70%). Điều này chứng minh rằng lỗi không nằm ở model hay thư viện – toàn bộ dữ liệu pixel gửi vào Edge Impulse bị sai định dạng do `int8_t` bị sign-extend khi cast sang `uint32_t`, phá hỏng giá trị kênh màu R và G trong mỗi pixel packed-RGB.

Độ chính xác tổng thể

Sau khi sửa lỗi, Edge Impulse EON trở thành back-end **vượt trội cả hai chiều**: accuracy cao hơn (80% vs 70%) *và* thời gian suy diễn nhanh hơn **32 lần** (203 ms vs 6453 ms). Điều này cho thấy EON compilation không chỉ tối ưu tốc độ mà còn giữ đúng chất lượng model khi dữ liệu đầu vào đúng định dạng.

Phân tích theo lớp – Edge Impulse

- **Metal đạt F1 = 1.000**: 4/4 mẫu được nhận diện chính xác với confidence rất cao (85–99%), không có false positive. Đây là lớp dễ nhất cho EI – kết cấu bề

mặt kim loại phẳng bóng tạo ra đặc trưng ảnh nhất quán.

- **Paper đạt F1 = 0.857 và Precision = 1.000**: 3/4 mẫu đúng, không có mẫu nào bị nhầm thành paper (FP = 0). Một mẫu (paper3) bị nhầm thành cardboard – cả hai đều là vật liệu phẳng màu trắng/nâu nhạt, khó phân biệt ở độ phân giải 96×96.

- **Biological là lớp có FP cao nhất (FP = 3)**: cả hai mẫu battery3 và battery4 *lẫn* cardboard4 đều bị nhầm thành biological. Nhìn vào confidence: battery3 (90.2%), battery4 (39.1%), cardboard4 (99.2%). Đặc biệt, cardboard4 bị nhầm với confidence rất cao – đây có thể là ảnh chụp cardboard bị ướt/bẩn, có màu nâu sũng nước tương đồng với vật liệu hữu cơ.

- **Battery có Recall = 0.500 (2/4)**: battery3 và battery4 bị nhầm thành biological. Nhiều khả năng đây là các viên pin tiểu/AA nằm trong môi trường có nền lá cây hoặc rác hữu cơ, khiến đặc trưng nền ảnh lấn át đặc trưng của pin.

Phân tích theo lớp – LiteRT

- **Metal đạt Recall = 1.000** (4/4 đúng) với confidence rất cao (96–99%), tuy nhiên Precision chỉ 0.500 vì toàn bộ 3 mẫu battery bị nhầm thành Metal (FP = 3 trên tổng 8 dự đoán Metal). Lỗi nhầm lẫn battery–metal là điểm yếu cốt lõi của back-end này.

- **Battery có Recall = 0.250 (1/4)**: battery2, battery3, battery4 đều bị nhầm thành Metal. Đây là hệ quả của việc model học đặc trưng bề mặt kim loại bóng từ dữ liệu train, làm mờ ranh giới giữa pin (cylindrical metal) và kim loại phẳng.

- **Biological, Paper và Cardboard** đạt F1 từ 0.750 đến 0.857 – tốt hơn nhiều so với Battery, cho thấy model phân biệt tốt các vật liệu phi kim loại.

- **Paper1 bị nhầm thành Cardboard (confidence 67.97%)**: đây là lỗi tương tự EI – hai lớp có bề mặt phẳng, màu sắc gần nhau. Confidence thấp (67%) gợi ý model cũng không hoàn toàn chắc chắn.

So sánh hai back-end

Edge Impulse EON vượt LiteRT cả về accuracy lẫn tốc độ sau khi sửa lỗi. Điểm yếu còn lại của EI là lớp biological bị overpredicted (Precision = 0.571). Điểm yếu của LiteRT là cặp battery–metal bị lẫn lộn nghiêm trọng. Hai back-end có cùng lỗi nhầm giữa paper và cardboard – đây là vấn đề của dữ liệu/model, không phải implementation.

Ảnh hưởng của thời gian suy diễn

Với 203 ms, Edge Impulse hoàn toàn đủ cho real-time (dưới 1 giây). Thời gian 6.45 giây của LiteRT không phù hợp cho phản hồi tức thời nhưng có thể chấp nhận cho IoT phân loại rác không yêu cầu real-time. Để tối ưu, có thể dùng model nhỏ hơn (MobileNetV1, EfficientNet-Lite0) hoặc bật CMSIS-NN kernels trên ARM Cortex-M.

Giới hạn của thử nghiệm

Bộ kiểm thử chỉ gồm 20 mẫu (4 mỗi lớp), nên mỗi sai lệch ảnh hưởng 25 điểm phần trăm đến metric. Kết quả mang tính tham khảo; cần ít nhất 50–100 mẫu/lớp để đánh giá tin cậy hơn.